

Prompt: 5 Fixes — Tablet, Edición Completa, Pago Checkout

Fix 1 — Modal de reserva desfasado en tablet

Problema

En tablet (768px-1024px) el `ReservationDetailModal` y `ReservationDetailDialog` se ven con el contenido cortado o fuera del dialog.

Fix en `ReservationDetailModal` (Planning)

Buscar:

```
<DialogContent className="sm:max-w-[500px]">
```

Reemplazar por:

```
<DialogContent className="w-[95vw] max-w-[560px] max-h-[90vh] overflow-y-auto">
```

Fix en `ReservationDetailDialog` (Reservas)

Buscar el `<DialogContent>` del componente `ReservationDetailDialog` y reemplazar su `className` por:

```
<DialogContent className="w-[95vw] max-w-[680px] max-h-[90vh] overflow-y-auto">
```

Fix general para todos los dialogs del sistema

Buscar todos los `sm:max-w-[500px]` y `sm:max-w-[600px]` en dialogs de reserva y reemplazar con el patrón `w-[95vw] max-w-[Xpx] max-h-[90vh] overflow-y-auto`. Esto asegura que en tablet el dialog ocupe el 95% del ancho disponible sin desbordarse.

Fix 2 — Botón "Editar" en planning abre form incompleto

Problema

El botón “Editar” en `ReservationDetailModal` (el modal del planning) abre un form simplificado con solo fechas, canal y observaciones. Debería abrir el `ReservationFormDialog` completo igual que desde la página de Reservas.

Causa

El modal del planning tiene su propio form de edición inline simplificado, en vez de reutilizar `ReservationFormDialog`.

Fix

Paso 1: En `PlanningPage`, agregar los estados necesarios para abrir el form completo:

```
// Agregar junto a los otros estados del planning
const [editReservationOpen, setEditReservationOpen] = useState(false);
const [editingReservationData, setEditingReservationData] = useState<ReservationW

// Queries necesarias para ReservationFormDialog (si no están ya en PlanningPage)
const { data: guests } = useQuery<Guest[]>({ queryKey: ["/api/guests"] });
const { data: rooms } = useQuery<RoomWithType[]>({ queryKey: ["/api/rooms"] });
const { data: roomTypes } = useQuery<RoomType[]>({ queryKey: ["/api/room-types"]
```

Paso 2: En `ReservationDetailModal`, agregar prop `onEdit`:

```
// Cambiar la firma del componente
function ReservationDetailModal({
  open,
  onOpenChange,
  reservationId,
  onNavigate,
  onEdit, // ← nuevo prop
}: {
  open: boolean;
  onOpenChange: (open: boolean) => void;
  reservationId: string | null;
  onNavigate: (path: string) => void;
  onEdit?: (reservation: ReservationWithDetails) => void; // ← nuevo
})
```

Paso 3: Reemplazar el botón “Editar” dentro de `ReservationDetailModal`.

Buscar el botón que actualmente abre el form simplificado (el que tiene `<Pencil>` o texto “Editar” en el `DialogFooter`) y reemplazarlo por:

```

{reservation && onEdit && (
  <Button
    variant="outline"
    onClick={() => {
      onEdit(reservation);
      onOpenChange(false);
    }}
    className="w-full sm:w-auto"
    data-testid="button-edit-reservation"
  >
    <Pencil className="h-4 w-4 mr-2" />
    Editar
  </Button>
)}

```

Paso 4: Donde se usa `ReservationDetailModal` en `PlanningPage`, pasar el nuevo prop y agregar el `ReservationFormDialog`:

```

<ReservationDetailModal
  open={reservationDetailOpen}
  onOpenChange={setReservationDetailOpen}
  reservationId={selectedReservationId}
  onNavigate={(path) => navigate(path)}
  onEdit={(reservation) => { // ← nuevo
    setEditingReservationData(reservation);
    setEditReservationOpen(true);
  }}
/>

{/* Form completo de edición – agregar al final de PlanningPage */}
{editingReservationData && (
  <ReservationFormDialog
    reservation={editingReservationData}
    guests={guests || []}
    rooms={rooms || []}
    roomTypes={roomTypes || []}
    open={editReservationOpen}
    onOpenChange={setEditReservationOpen}
    onSuccess={() => {
      setEditingReservationData(null);
      queryClient.invalidateQueries({ queryKey: ["/api/planning"] });
    }}
  />
)}

```

Fix 3 — Desde “Reservas” el modal no tiene botón Editar

Problema

`ReservationDetailDialog` (el modal que se abre al hacer clic en “Ver Detalle” desde la lista de Reservas) no tiene botón “Editar”. Solo tiene “Anular Reserva” y “Cerrar”.

Fix

Paso 1: Agregar prop `onEdit` a `ReservationDetailDialog`:

```
function ReservationDetailDialog({
  reservation,
  open,
  onOpenChange,
  onCancel,
  onEdit,    // ← nuevo
}): {
  reservation: ReservationWithDetails;
  open: boolean;
  onOpenChange: (open: boolean) => void;
  onCancel: () => void;
  onEdit?: () => void;    // ← nuevo
})
```

Paso 2: Agregar el botón en el footer del dialog (junto al botón “Anular Reserva”):

```
/* Buscar el footer con el botón Anular y agregar Editar */
<div className="flex gap-2 flex-wrap">
  {onEdit && (
    <Button
      variant="outline"
      onClick={() => {
        onEdit();
        onOpenChange(false);
      }}
      data-testid="button-edit-from-detail"
    >
      <Pencil className="h-4 w-4 mr-2" />
      Editar Reserva
    </Button>
  )}
  <Button variant="destructive" onClick={onCancel}>
    <X className="h-4 w-4 mr-2" />
```

```

        Anular Reserva
    </Button>
    <Button variant="outline" onClick={() => onOpenChange(false)}>
        Cerrar
    </Button>
</div>

```

Paso 3: En la página de Reservas, pasar el prop `onEdit` al `ReservationDetailDialog`:

```

<ReservationDetailDialog
  reservation={selectedReservation}
  open={detailDialogOpen}
  onOpenChange={setDetailDialogOpen}
  onCancel={() => setCancelDialogOpen(true)}
  onEdit={() => {                                // ← nuevo
    setDetailDialogOpen(false);
    setDialogOpen(true);                          // abre el ReservationFormDialog
  }}
/>

```

Fix 4 — Check-in no disponible en algunas reservas

Problema

Algunas reservas no muestran el botón de Check-in aunque sea el día correcto. El problema es de `timezone`: `new Date().toISOString().split("T")[0]` devuelve la fecha en UTC, que en Argentina (UTC-3) puede ser el día anterior.

Fix — Función helper de fecha local

Agregar esta función utilitaria (puede ir en un archivo `utils.ts` o directamente en el componente):

```

// Fecha de hoy en formato YYYY-MM-DD usando hora local (no UTC)
function getTodayLocal(): string {
  const now = new Date();
  return `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, '0')}-${S
}

```

Fix en `ReservationDetailModal` (`canCheckIn`)

Buscar:

```
const canCheckIn = reservation?.status === "confirmed" || reservation?.status ===
```

Reemplazar por:

```
const todayLocal = getTodayLocal();
const canCheckIn = (
  reservation?.status === "confirmed" || reservation?.status === "pending"
) && reservation?.checkInDate <= todayLocal;
```

Fix en `ReservationDetailDialog` **y página Check-in**

Aplicar el mismo patrón `getTodayLocal()` en cualquier lugar donde se compare fechas de check-in con `new Date().toISOString().split("T")[0]`.

Nota: Si se quiere permitir check-in anticipado (early check-in), la condición es:

```
const canCheckIn = (
  reservation?.status === "confirmed" || reservation?.status === "pending"
) && (reservation?.checkInDate <= todayLocal || reservation?.earlyCheckIn === true)
```

Fix 5 — “Registrar Pago” en checkout no avanza

Problema

En el paso 2 del wizard de checkout (Pago), al hacer clic en “Registrar Pago” el botón no hace nada visible. La mutation `addPaymentMutation` no tiene handler de error, entonces si falla silenciosamente el usuario no sabe qué pasó.

Causa probable

El endpoint `POST /api/payments` recibe `reservationId` pero puede estar fallando si el `folio` no existe todavía para esa reserva, o si `amount` llega como string vacío.

Fix 1 — Agregar `onError` a `addPaymentMutation`

Buscar `addPaymentMutation` en el componente de checkout (cerca de la línea que tiene `refetchFolio()` en el `onSuccess`) y agregar el handler de error:

```
const addPaymentMutation = useMutation({
  mutationFn: async (data: { amount: string; method: PaymentMethod; reference: string }) => {
    return apiRequest("POST", "/api/payments", {
```

```

        reservationId: selectedReservation!.id,
        amount: data.amount,
        method: data.method,
        date: new Date().toISOString().split("T")[0],
        reference: data.reference || null,
    });
},
onSuccess: () => {
    refetchFolio();
    setPaymentAmount("");
    setPaymentReference("");
    toast({ title: "Pago registrado" });
},
onError: (error: any) => { // ← agregar esto
    const message = error?.data?.error || error?.message || "No se pudo registrar
    toast({
        title: "Error al registrar pago",
        description: message,
        variant: "destructive",
    });
    console.error("Payment error:", error);
},
});

```

Fix 2 — Validar amount antes de llamar la mutation

En el botón “Registrar Pago”, asegurar que el monto sea válido:

```

<Button
  onClick={() => {
    const amount = paymentAmount || balance.toFixed(2);
    if (!amount || parseFloat(amount) <= 0) {
      toast({ title: "Ingresá un monto válido", variant: "destructive" });
      return;
    }
    addPaymentMutation.mutate({
      amount,
      method: paymentMethod,
      reference: paymentReference,
    });
  }}
  disabled={addPaymentMutation.isPending}
  data-testid="button-register-payment"
>
  {addPaymentMutation.isPending ? "Registrando..." : "Registrar Pago"}
</Button>

```

Fix 3 — Verificar endpoint `/api/payments` en backend

En `routes.ts`, buscar `app.post("/api/payments")` y verificar que:

1. Crea el folio si no existe antes de registrar el pago
2. Acepta `amount` como string y lo convierte con `parseFloat()`
3. Devuelve error descriptivo si falla

Si el endpoint no crea el folio automáticamente, agregar:

```
// Antes de insertar el pago, asegurar que existe el folio
let folio = await storage.getFolioByReservation(reservationId);
if (!folio) {
  folio = await storage.createFolio({ reservationId, status: "open" });
}
```

Resumen de cambios

Fix	Archivos afectados
1 - Responsive tablet	<code>ReservationDetailModal</code> , <code>ReservationDetailDialog</code>
2 - Editar completo desde planning	<code>PlanningPage</code> , <code>ReservationDetailModal</code>
3 - Botón editar en Reservas	<code>ReservationDetailDialog</code> , página Reservas
4 - Check-in fechas timezone	<code>ReservationDetailModal</code> , <code>ReservationDetailDialog</code> , <code>getTodayLocal()</code>
5 - Registrar pago no avanza	Componente checkout wizard , <code>routes.ts</code>